Índice

A

a11y (véase accesibilidad (a11y))
accesibilidad (a11y), ¿Qué es la usabilidad?¿Qué es la accesibilidad?
  – (véase también usabilidad)
  – retos abordados por, ¿Qué es la accesibilidad?
  – leyes y normativas, Diseño para todos
Propiedades ACID (véase Propiedades de atomicidad, consistencia, aislamiento y durabilidad (ACID)).
ADA (véase Ley de Estadounidenses con Discapacidades (ADA))
ADR (véase Registros de decisiones arquitectónicas (ADR))
Estándar de cifrado avanzado (AES), Cifrado: tu última línea de defensa
Programación agencial (véase codificación vibe)
IA (véase Inteligencia artificial (IA))
Ingeniero de software impulsado por IA, La IA como multiplicador de tu fuerza: de escribir código a resolver problemas
  – Multiplicador de fuerzas, la IA como tu multiplicador de fuerzas: de escribir código a resolver problemas
Ley de Estadounidenses con Discapacidades (ADA), Diseño para todos

– (véase también accesibilidad (a11y))
Arquitecto, tipos de,Compensaciones-Compensaciones, El arquitecto ágil-El arquitecto ágil
– Arquitecto accidental,Compensaciones-Compensaciones
– Arquitecto ágil, Elarquitecto ágil:el arquitecto ágil
– desarrolladores sénior como arquitectos, El arquitecto ágil
registros de decisiones arquitectónicas (ADR), cómo abordar código desconocido, revisión de la documentación disponible,registros de decisionesarquitectónicas-registros de decisiones arquitectónicas
– ejemplos, registros de decisiones arquitectónicas
– Plantilla, Registros de decisiones arquitectónicas
patrones arquitectónicos, Comprensión de la arquitectura y la estructura del proyecto
– Arquitectura hexagonal (puertos y adaptadores), Comprensión de la arquitectura y la estructura del proyecto
– Paquete por característica, Comprensión de la arquitectura y la estructura del proyecto
– Paquete por capa, Comprensión de la arquitectura y la estructura del proyecto
Estilos arquitectónicos, cómo lograrla alineación de las partes interesadas:estilos arquitectónicos
– Monolítico frente a distribuido, estilos arquitectónicos
Inteligencia artificial (IA), la práctica hace al maestro, elingeniero de software impulsado por IA: resumen
– Capacidades y limitaciones, Comprensión de las capacidades ylimitaciones de la IA:limitaciones actuales de la IA
– Limitaciones de la IA:limitaciones actuales de la IA

  - Fortalezas de la IA: en qué destaca la IA para los desarrolladores de software:en qué destaca la IA para los desarrolladores de software
- Como multiplicador de fuerzas, la IA como multiplicador de fuerzas: de escribir código a resolver problemas
- Para generar diagramas, crea modelos mentales de forma incremental
  - (véase también diagramas como código)
- Para generar datos de prueba, datos estructurados
- Impacto en la ingeniería de software: Cómo la IA podría dar forma ala ingeniería de software: La IA como multiplicador de fuerzas: de escribir código a resolver problemas
- para la preparación de entrevistas, investigación estratégica, la práctica hace al maestro
- Como acelerador del aprendizaje, Elhábito de aprender:aprender a través de la IA
- para monitorear, monitoreo
- como programador en pareja, la IA como tuprogramador en pareja:entornos IDE de IA agencial
  - (véase también programación en pareja)
  - entorno IDE de IA agencial, entornos IDE de IA agencial
  - asistente IDE en línea, Asistentes IDE en línea
  - asistente chatbot independiente, Asistentes chatbot independientes
- ingeniería de prompts,Fundamentos de la ingeniería de prompts:técnicas organizativas
  - prompts de pocos disparos, técnicas de estructuración

– meta-prompting, consejos prácticos para una mejora inmediata
– Prompts de un solo disparo, técnicas de estructuración
– Técnicas,técnicas esenciales de ingeniería de prompts:técnicas organizativas
– Prompting de cero disparos, técnicas de estructuración
– Riesgo de alucinación, la práctica hace al maestro
– papel en la comprensión del código, la práctica hace al maestro
– Terminología, desmitificandola terminología de la IA:grandes modelos de lenguaje
– Deep Learning, DeepLearning-Generative AI
– IA generativa (GenAI),IA generativa:IA generativa
– Modelos de lenguaje grandes (LLMs), Modelos de lenguaje grandes
– aprendizaje automático (ML), aprendizaje automático:Deep Learning
– para escribir pruebas, Introducción
aserciones, Aserciones
Confirmaciones atómicas, Pequeñas, cambios reversibles, confirma estratégicamente
Propiedades de atomicidad, consistencia, aislamiento y durabilidad (ACID), bases de datos relacionales
Pruebas automatizadas,Pruebas automatizadas: resumen
– Ventajas de las pruebas automatizadas:conducen a la coherencia y la repetibilidad
– Aumento de la confianza, Aumenta tuconfianza: aumenta tu confianza
– (véase también pruebas de regresión)

– Coherencia y repetibilidad, conduce a la coherencia yla repetibilidad: conduce a la coherencia y la repetibilidad
– (véase también pruebas manuales)
– Mejora la mantenibilidad, mejorala mantenibilidad: mejora la mantenibilidad
– (véase también principio de responsabilidad única (SRP))
– simulación, simulación
– tipos de, Tipos depruebas automatizadas:pruebas de extremo a extremo
– pruebas de extremo a extremo (E2E), Pruebas de extremo a extremo
– Pruebas de integración, Pruebas de integración
– pirámide de pruebas, Tipos depruebas automatizadas:tipos de pruebas automatizadas
– Pruebas unitarias, Pruebas unitarias
– Qué evitar, Qué no debes probar
AWS Secrets Manager, secretos
Azure Key Vault, secretos

B

Notación O grande, Adopta la filosofía del programador perezoso
Paradoja de Blub, sesgos cognitivos
– (véase también sesgos cognitivos)
Pruebas frágiles, pruebas de extremo a extremo
– (véase también falsos negativos)
Desarrollo brownfield, el reto de trabajar con código existente
– (véase también proyectos greenfield; código heredado)

C

Modelo C4, ¿Qué es el modelado de software y por qué lo hacemos?
almacenamiento en caché, elegir elmodelo de consistencia adecuado: almacenamiento encaché y consideraciones de consistencia
- Consideraciones junto con la consistencia, estrategias de almacenamiento en caché y consistencia
- Estrategias para el almacenamiento en caché:estrategias comunes de almacenamiento en caché:cuándo utilizar el almacenamiento en caché, optimización del rendimiento
  - almacenamiento en caché aparte (carga diferida), estrategias comunes de almacenamiento en caché
  - escritura diferida (write-back), estrategias comunes de almacenamiento en caché
  - escritura directa, estrategias comunes de almacenamiento en caché

Ley de Privacidad del Consumidor de California (CCPA), requisitos de cumplimiento
Teorema CAP, teorema CAP y sus implicaciones
- (véase también modelos de consistencia)
- Sistemas AP, teorema CAP y sus implicaciones
- Sistemas CA, teorema CAP y sus implicaciones
- Sistemas CP, teorema CAP y sus implicaciones

Cardinalidad, diagramas de clases
gestión de la carrera profesional,gestión de la carrera profesional: resumen
- Cómo trazar tu camino,consideraciones prácticas:retos del desarrollo de software a distancia

  - Creación de una comunidad profesional, reconocimiento de oportunidades de crecimiento:mantén y participa en tu red
    - (véase también habilidades sociales)
  - Síndrome del impostor, superarlo, celebrar y registrar tuslogros: reconocer las oportunidades de crecimiento
  - entrevistas, Mantener y participar en tu red:crecer a partir de los comentarios
  - Equilibrio entre la vida laboral y personal, crecer a partir delos comentarios: retos del desarrollo remoto de software
- Planificar tu trayectoria, gestión de la carrera profesional:consideraciones prácticas
  - Trayectorias alternativas, Trayectorias alternativas
  - Opciones profesionales, exploración, experimentar con proyectos paralelos:trayectorias alternativas
  - Elegir el tipo de empresa adecuado, Tipos de empresas y su adecuación a la vida laboral y personal
  - Factores a tener en cuenta, Alinear las elecciones profesionales conlas etapas de la vida:consideraciones prácticas
  - Encuentra tu pasión, planifica tu trayectoria profesional:experimenta con proyectos paralelos
  - establecimiento de objetivos (trabajar hacia atrás),Caminos alternativos:retroceder desde tus objetivos
  - Trayectoria de liderazgo, Trayectoria de liderazgo
  - Adquisición de habilidades, deliberada, Retroceder desde tusobjetivos: crea un radar tecnológico personal
    - (véase también aprendizaje)
  - Trayectoria de experto técnico, Trayectoria de experto técnico

CCPA (véase Ley de Privacidad del Consumidor de California (CCPA))
CDC (véase captura de datos de cambio (CDC))
CDN (véase redes de distribución de contenidos (CDN))
captura de datos modificados (CDC), sincronización de datos
Estrategia de gestión del cambio, Estrategias de gestión del cambio
CI/CD (véase integración continua y implementación continua (CI/CD))
Clear, James, Staying on Task
código como diseño, ¿Qué es el modelado de software y por qué lo hacemos?
Comentarios en el código, El problema con los comentarios en el código:el problema con los comentarios en el código
– (véase también el principio «No te repitas» (DRY); pruebas como documentación)
– como código sospechoso, El problema con los comentarios en el código
cobertura de código,Cobertura de código: cobertura de código
– como métrica vanidosa, Abordar código desconocido, Cobertura de código
kata de código, Poniéndolo en práctica
revisiones de código, Evita el código ingenioso:fomenta la confianza, revisiones de código Vibe
– (véase también programación en pareja)
– de la inteligencia artificial (IA), Por qué las revisiones de código son más importantes que nunca
– revisión de código con casillas de verificación (antipatrón), Revisiones de código

– Pirámide de revisión de código, Revisiones de código
– Empatía, Evita la revisión de código con casillas de verificación
– Fomentar la confianza, es difícil ser criticado
– (véase también propiedad colectiva del código)
– Respuesta «a mí me parece bien» (LFTM), Evita la revisión de código por casillas
Olores de código, ¿Qué es un buen código?
– (véase también código bueno, definición de)
programador, definición de (véase programador, definición de)
sesgos cognitivos, El reto de trabajar concódigo existente:sesgos cognitivos
– Efecto IKEA, sesgos cognitivos
– Efecto de mera exposición, sesgos cognitivos
propiedad colectiva del código, fomentar la confianza
– (ver también revisiones de código)
Complejidad, evita el código ingenioso
– Complejidad accidental, evita el código ingenioso
– Complejidad esencial, evita el código ingenioso
Principio de composición sobre herencia, cuidado con el código repetitivo
Grupo de conexiones, conexiones a bases de datos
Modelos de consistencia, modelos de consistencia:elegir el modelo de consistencia adecuado
– (véase también el teorema CAP)
– Tiposde modelos de consistencia
Contratos impulsados por el consumidor, Pruebas como documentación
– (véase también pruebas como documentación)

– Herramientas para: Pruebas como documentación
– Pact, Pruebas como documentación
– Spring Cloud Contract, Pruebas como documentación
Contenedorización, «Pero funciona en mi máquina»
redes de distribución de contenidos (CDN), optimización del rendimiento
integración continua e implementación continua (CI/CD), Integración continua eimplementación continua:patrones avanzados de CI/CD
– (véase también automatización de la implementación)
– GitHub Actions, Creación de un flujo de trabajo básico de CI/CD
– flujo de trabajo, Creación de un flujo de trabajo básico de CI/CD

D

Datos, trabajar condatos: resumen
– (véase también migración de datos; tipos de datos; bases de datos)
– Consideraciones especializadas paraconjuntos de datos grandes
YAML
– datos binarios, Consideraciones especializadas sobre datos
– Datos de fecha y hora, Datos binarios: datos de fecha y hora
– conjuntos de datos grandes, Conjuntos de datos grandes-Conjuntos de datos grandes
formatos de datos,datos no estructurados:YAML
– CSV (valores separados por comas),XML-CSV
– JSON (JavaScript Object Notation),Formatos de datos comunes:JSON
– XML (lenguaje de marcado extensible), JSON

– YAML (Yet Another Markup Language), YAML
Migración de datos, equilibrio entre complejidad yrendimiento: gestión delas dependencias y transformaciones de datos
– Migración radical frente a migración por fases, comprender los fundamentos del movimiento de datos
– Dependencias y transformaciones de datos, gestión, gestión de dependencias y transformaciones de datos
– Sincronización de datos, sincronización de datos
– Procesos de extracción, transformación y carga (ETL), migración radical frente a migración por fases
– Cambios en el esquema, gestión,sincronización de datos:gestión delas dependencias y transformaciones de datos
Persistencia de datos, patrones para, Persistencia ygestión de datos:mapeo relacional deobjetos
– Acceso directo a bases de datos, Persistencia y gestión de datos
– mapeo relacional de objetos (ORM), mapeo relacional de objetos
– patrón de repositorio, patrón de repositorio
tipos de datos,datos estructurados frente a datos no estructurados:datos no estructurados
– datos estructurados,datos estructurados frente a datos no estructurados:datos estructurados
– características de, datos estructurados
– datos no estructurados,datos estructurados frente adatos no estructurados
– Características de los datos no estructurados
bases de datos, Almacenamientoeficaz de datos: almacenamientoen caché y consistencia
– modelos de consistencia (véase modelos de consistencia)

  - Transacciones
    - (véase también Propiedades de atomicidad, consistencia, aislamiento y durabilidad (ACID))
  - Tipos debases de datosvectoriales
    - Bases de datos de documentos, bases de datos de documentos
    - bases de datos de grafos,bases de datos de grafos:bases de datos vectoriales
    - almacenamientos clave-valor, Almacenamientos clave-valor
    - Bases de datos relacionales, Tipos de bases de datos y sus casos de uso:bases de datos de documentos
    - Bases de datos vectoriales,bases de datos vectoriales:bases de datos vectoriales

canalización de implementación, Canalización de implementación:patrones avanzados de CI/CD

  - Automatización de la implementación, Automatización de la implementación:procedimientos de reversión
    - (véase también integración continua e implementación continua (CI/CD))
  - Entornos de implementación, entornos de implementación
  - Estrategias de implementación, Estrategias de implementación:elegir tu estrategia
    - implementación total (big bang), Implementación total (big bang)
    - implementación azul-verde, consideraciones sobre el tiempo de inactividad cero
    - implementación canaria, Implementación gradual (enfoque por fases)

– Implementación gradual (por fases), Implementación gradual (enfoque por fases)
– Implementación continua, Implementación gradual (en fases)
Principios de diseño (CRAP), Maximización dela usabilidad-proximidad
– Alineación, repetición
– ejemplo aplicado, aplicación de los principios dediseño: aplicación de los principios de diseño
– Contraste, Principios de diseño
– proximidad,alineación-proximidad
– repetición, contraste
Acciones destructivas,Acciones destructivas:conclusión
acciones destructivas, diseño de medidas de protección para,Acciones destructivas: conclusión
Desarrollador, definición de, Un ingeniero con cualquier otro nombre
Diagramas, ¿Qué es el modelado de software y por qué lo hacemos? -Diagramas adicionales
– (véase también modelado de software)
– para la comunicación arquitectónica, diagramas arquitectónicos
– diagramas de clases, diagramas de clases
– Diagramas de componentes, Diagramas de componentes
– Diagramas de contexto, Diagramas de contexto
– modelos de datos, Modelos de datos
– diagramas de implementación, diagramas de implementación
– Diagramas de seguridad, Diagramas de implementación
– modelos de recuperación ante desastres, diagramas adicionales

– como artefactos efímeros, ¿Qué es el modelado de software y por qué lo hacemos?
– tormenta de eventos, diagramas adicionales
– formal frente a informal, ¿Qué diagramas necesitas?
– Diagramas de secuencia, diagramas de secuencia
– mapeo de historias de usuario, diagramas adicionales
– Mapeo de la cadena de valor, diagramas adicionales
Definición de tipo de documento (DTD), XML
documentación, cómo abordar código desconocido
– La trampa de la documentación, revisión de la documentación disponible
– para código desconocido, cómo abordar código desconocido
– Verificación con el código, revisión de la documentación disponible
Principio «No te repitas» (DRY), El problema de los comentarios en el código
DTD (véase Definición de tipo de documento (DTD))

E

Casos de perímetro, Descubrir casos de perímetro y límites
Pruebas de extremo a extremo (E2E), Pruebas de extremo a extremo
Mensajes de error, la importancia de unos buenos mensajes de error:acciones destructivas
– Diseño eficaz de, La importancia de los buenos mensajes de error:acciones destructivas
Formularios propensos a errores, Evitar el código inteligente
Ley Europea de Accesibilidad (EAA), Diseño para todos

– (véase también accesibilidad (a11y))
Excepciones, engañosas, Arqueología del software
Flujo de ejecución, comprender el flujo de ejecución:trabajar con marcos internos
– Puntos de entrada de la aplicación, Encontrarpuntos de entrada de la aplicación:encontrar puntos de entrada de la aplicación
– puntos de entrada comunes, Encontrar puntos de entrada de aplicaciones
– Seguimiento de solicitudes, Seguir los datos: seguimiento de los recorridos de las solicitudes:depuración
– Herramientas de prueba de API, Herramientas de prueba de API
– Herramientas de desarrollo del navegador, herramientas de desarrollo del navegador
– Depuración, Depuración
– Registro, Registro

F

Falsos negativos, pruebas de extremo a extremo
– (véase también pruebas frágiles)
miedo a perderse algo (FOMO), miedo a perderse algo
funciones de aptitud,Funciones deaptitud-Funciones de aptitud
– (véase también arquitectura evolutiva)
Los cinco porqués, no resuelvas demasiado rápido,no resuelvas demasiado rápido
Flyway, uso del control de versiones para estructuras de datos
– (véase también migración de datos)
FOMO (véase miedo a perderse algo (FOMO))

Perspectiva nueva, valor de, El valor de una mirada nueva
– (véase también la mentalidad «siempre lo hemos hecho así»)
Funciones como servicio, menos es más

G

Reglamento General de Protección de Datos (RGPD), requisitos de cumplimiento
GitHub Actions, Creación de un flujo de trabajo básico de CI/CD
regla de oro del software, aplicar la regla de oro al software, mensajería
– Optimización para los seres humanos, aplicar la regla de oro al software
Código bueno, definición de, no reinventes la rueda
– (véase también «code smells»)
Proyectos nuevos, lo que te enseñaron frente a lo que necesitas saber, el reto de trabajar con código existente
– (véase también código heredado)

H

HashiCorp Vault, secretos
Efecto Hawthorne, ¿Qué es un buen código?
Ley de Portabilidad y Responsabilidad del Seguro Médico (HIPAA), requisitos de cumplimiento
Código heredado (véase código heredado)
HTTPS (HTTP Secure), proteger las comunicaciones con HTTPS
Protocolo de transferencia de hipertexto (HTTP), Seguridad de las comunicaciones con HTTPS

Desarrollo basado en hipótesis, El arquitecto ágil
– (véase también arquitectura de software)

I

I18N (véase internacionalización (I18N))
Ibilidades (véase atributos de calidad)
Entornos de desarrollo integrados (IDE), Aprovechamiento de las funciones del IDE:visualización de la estructura del código
– Funciones de análisis de código, Funciones de análisisde código: visualización de la estructura del código
– herramientas de navegación por el código, herramientas de navegación por el código: jerarquías de llamadas y vistas de dependencias
– jerarquías de llamadas, Jerarquías de llamadas y vistas de dependencias
– Búsqueda de usos y referencias, Búsqueda de usos y referencias
– Saltar a la definición, Saltar a la definición
Intención frente a implementación, escribir código para que sea legible
– (véase también convenciones de nomenclatura)
Internacionalización (I18N), ¿Qué son la localización y la internacionalización?
– (véase también localización (L10N))
– Compatibilidad de derecha a izquierda (RTL), ¿Qué son la localización y la internacionalización?
Método de pirámide invertida, para escribir código, Escribe código para que se lea

problema «funciona en mi máquina», «Pero funciona en mi máquina»
– (véase también entorno de producción, complejidades de)
– razones para «Pero funciona en mi máquina»
– soluciones para «Pero funciona en mi máquina»

J

Interfaz Java, buscar usos y referencias
JUnit, Introducción
– (véase también escribir pruebas)

L

L10N (véase localización (L10N))
Aptitud lingüística frente a aptitud matemática, Los muchos caminos para convertirse en ingeniería de software, Aprender nuevos idiomas
Modelos de lenguaje grandes (LLMs), Modelos de lenguaje grandes
La filosofía del programador perezoso, lo que te enseñaron frente a lo que necesitas saber:adopta la filosofía del programador perezoso
– frente al enfoque de fuerza bruta, Adopta la filosofía del programador perezoso
– Ejemplo de asignación de mayúsculas, Adopta la filosofía del programador perezoso
aprendizaje, aprender aaprender: resumen
– El agotamiento, cómo evitarlo, sesiones informativas sobre arquitectura
– evaluar nuevas tecnologías, miedo a perderse algo

  - En entornos heredados, sesiones informativas sobre arquitectura
- miedo a perderse algo (FOMO), aprender a través de la IA:miedo a perderse algo
- Cómo funciona el cerebro, Aprender aaprender: estudiar a última hora no funciona
  - Modos de procesamiento cerebral (modo R/modo L), el empollar no funciona
  - empollar, ineficacia de, empollar no funciona
- Innovación, practicar en las organizaciones, ¿Dónde debes invertir tu tiempo?-Practicar la innovación
  - Reuniones informativas sobre arquitectura, Practicala innovación: reuniones informativas sobre arquitectura
  - Clubes de lectura, practica la innovación
  - hackatones, practica la innovación
- Hábitos de aprendizaje,Adquisición de habilidades:el hábito de aprender
  - Estrategia de aprendizaje profundo, El hábito de aprender
  - rutina del café matutino, El hábito de aprender
  - Repetición espaciada, El hábito de aprender
- caminos de aprendizaje, un ingeniero con cualquier otro nombre:los muchos caminos para convertirse en ingeniería de software
  - campamentos de entrenamiento, Los muchos caminos para convertirse en ingeniería de software
  - Títulos en informática, Los muchos caminos para convertirse en ingeniería de software

– autodidacta, Los muchos caminos para convertirse en ingeniería de software
– nuevos idiomas, Fomentarla confianza: aprender nuevos idiomas
– La paradoja de«lo nuevo y brillante»: aprender a aprender
– Modelos de adquisición de habilidades, el empollamiento no funciona:adquisición de habilidades
– Modelo de Dreyfus,Adquisición dehabilidades: adquisición de habilidades
– Imitar, asimilar, innovar, adquisición de habilidades
– Shu Ha Ri, adquisición de habilidades
Código heredado, lo que te enseñaron frente a lo que necesitas saber, el reto de trabajar concódigo existente:sesgos cognitivos
– (véase también proyectos greenfield)
– Problemas que resolver, El reto de trabajar con código existente
Aprovechar las soluciones existentes, escribir código:no reinventar la rueda
– (véase también la filosofía del programador perezoso)
– bibliotecas que revisar, No reinventes la rueda
– Apache Commons, no reinventes la rueda
– Google Guava, no reinventes la rueda
LLMs (véase grandes modelos de lenguaje (LLMs))
localización (L10N), ¿Qué son la localización y la internacionalización?
– (véase también internacionalización (I18N))

M

aprendizaje automático (ML), aprendizaje automático

Pruebas manuales, conducen a la coherencia y la repetibilidad
– (véase también pruebas automatizadas)
Modelos mentales, crea modelos mentales de forma incremental
– desglosar sistemas complejos, desglosar sistemas complejos
– construcción incremental, Construye modelos mentales de forma incremental
– Visualización, visualizar tus modelos mentales:ampliar gradualmente tu alcance
métricas, para la calidad del código, ¿Qué es un buen código?- ¿Qué es un buen código?
– Complejidad ciclomática, ¿Qué es un buen código?
– Analizadores de código fuente, ¿Qué es un buen código?
– JSHint, ¿Qué es un buen código?
– PMD, ¿Qué es un buen código?
– SonarQube, ¿Qué es un buen código?
Microservicios, menos es más, comprender la arquitectura y la estructura del proyecto, ¿qué es la arquitectura?, estilos arquitectónicos, ¿dónde debes invertir tu tiempo?
ML (véase aprendizaje automático (ML))
burla,burlarse
– como doble de acción, Mocking
Mockito, simulación
– (véase también simulación)
MockMvc, redacción de pruebas de extremo a extremo
modelado, buenas prácticas, buenas prácticas de modelado:establecer estándares y plantillas
– Elección de colores en diagramas, ten cuidado con la elección de colores

– Simplicidad, manténla simplicidad
– Estándares y plantillas: establece estándares y plantillas
– público objetivo, conoce a tu público
modelado, herramientas para,Herramientas: herramientas
– Pizarras colaborativas, herramientas
– Lucidchart, herramientas
– Miro, herramientas
– Mural, Herramientas
– Diagramas como código, Herramientas
– Mermaid, herramientas
– PlantUML, herramientas
– Structurizr, Herramientas
– Microsoft Visio, Herramientas
– OmniGraffle, Herramientas
– Software de presentación, Herramientas
monitoreo,Monitoreo-Monitoreo
– (véase también observabilidad, registro)

N

convenciones de nomenclatura, escribir código para ser leído
– juego gibberish, Escribir código para ser leído
procesamiento del lenguaje natural (NLP), datos no estructurados
Nygard, Michael, Registros de decisiones arquitectónicas

O

mapeo relacional de objetos (ORM), mapeo relacional de objetos

Observabilidad, uso deplanes de ejecución de consultas:monitoreo y análisis de bases de datos, problema de consulta «Pero funciona en mi máquina»
– Problema de consulta N+1, pilares del monitoreo y el análisis de bases de datos
– Pilares del monitoreo y el análisis de bases de datos
– registro, monitoreo y análisis de bases de datos
– métricas, monitoreo y análisis de bases de datos
– Seguimiento, monitoreo y análisis de bases de datos
Proyecto de Seguridad de Aplicaciones Web Abiertas (OWASP), Fundamentos de seguridad
ORM (véase mapeadores relacionales de objetos (ORM))
ingeniería excesiva, no te precipites en encontrar soluciones, conclusión
– (véase también solución prematura)
OWASP (véase Proyecto Abierto de Seguridad de Aplicaciones Web (OWASP))

P

programación en pareja, revisiones de código, aprendizaje colaborativo
– (véase también revisiones de código)
Principio de Pareto, gestión de errores y registro
Estándar de seguridad de datos de la industria de tarjetas de pago (PCI DSS), requisitos de cumplimiento
gestión del conocimiento personal (PKM),gestión del conocimiento-personal-gestión del conocimiento personal
– (véase también productividad)

– Estrategias para tomar notas, gestión del conocimiento personal
– herramientas para, gestión del conocimiento personal
Información de identificación personal (PII), Protección de los datos de los usuarios
función de pellizcar para ampliar, hacer que lo correcto sea lo obvio
– (véase también usabilidad)
PKM (véase gestión del conocimiento personal (PKM))
PoC (véase prueba de concepto (PoC))
Poder de la indirección, lograr la alineación de las partes interesadas
– (véase también alineación de las partes interesadas)
Principios de diseño (véase principios de diseño (CRAP))
problemas de conceptos erróneos, no soluciones demasiado rápido, gestión del tiempo
entorno de producción, aproducción: resumen
– Complejidades de la producción:«Pero funciona en mi máquina»
– diferencias entre entornos, «Pero funciona en mi máquina»
– (véase también el problema «funciona en mi máquina»)
– imprevisibilidad de los usuarios, Los usuarios son imprevisibles:los usuarios son imprevisibles
Código listo para producción, Creación, «Pero funciona en mi máquina»:Requisitos de cumplimiento
– gestión de la configuración,configuraciones específicas del entorno:secretos
– archivos de configuración, archivos de configuración
– Variables de entorno, Variables de entorno
– Indicadores de características, Indicadores de características

– (véase también versiones canarias)
– gestión de secretos, secretos
– gestión de errores, Gestión de errores
– registro, Registro
– (véase también gestión de errores, monitoreo)
– optimización del rendimiento,optimización delrendimiento-optimización del rendimiento
productividad, Aumenta tu productividad
– Concepto de desarrollador 10x, Aumenta tu productividad
– automatización, estratégica, Automatización estratégica
– scripts de ayuda, automatización estratégica
– aprendizaje colaborativo, aprendizaje colaborativo
– Interfaz de línea de comandos(CLI), Aprovecha el poder de la línea de comandos:Aprovecha el poder de la línea de comandos
– herramientas para, Aprovecha el poder de la línea de comandos
– entorno de desarrollo, optimización, Optimizar tu entorno de desarrollo:aprovecha el poder de los atajos de teclado
– personalización de, Crea tu propio sable láser
– dominio de los IDE, conoce tus herramientas de desarrollo
– (véase también entornos de desarrollo integrados (IDE))
– entorno físico (ergonomía), construye tu propio sable láser
– estado de flujo, conoce tus herramientas de desarrollo
– Atajos de teclado, dominio, Aprovecha el poder delos atajos de teclado:Aprovecha el poder de los atajos de teclado
– gestores de portapapeles, Aprovecha el poder de los atajos de teclado

– Expansores de texto, Aprovecha el poder de los atajos de teclado
– mecanografía táctil, Aprovecha el poder de los atajos de teclado
Transición de programador a ingeniero, De programador aingeniero: resumen
programador, definición de, un ingeniero con cualquier otro nombre
La programación como comunicación, escribir código
– (véase también la regla de oro del software)
prueba de concepto (PoC), Big Bang frente a migración por fases
solicitudes de extracción (PR), evitar la revisión de código por casillas, solicitudes de extracción y revisiones de código
– (véase también revisiones de código)

Q

atributos de calidad,atributos de calidad: conseguirla alineación de las partes interesadas
– (véase también arquitectura de software)
– Comparación de conciertos kata, Identificar atributos de calidad
– relaciones inversas entre, atributos de calidad
– requisitos no funcionales, atributos de calidad
– alineación de las partes interesadas, identificación de atributos de calidad:obtención dela alineación de las partes interesadas
– (véase también poder de la indirección)
Optimización de consultas, consulta y gestióndel rendimiento de los datos:equilibrio entre complejidad y rendimiento
– optimización básica, optimización básica de consultas

– índices de bases de datos, sentencias preparadas
– Paginación, gestión deconjuntos de resultados grandes:herramientas y buenas prácticas
– sentencias preparadas,optimización básica de consultas:sentencias preparadas
Planificadores de consultas (optimizadores), herramientas y buenas prácticas:uso de planes de ejecución de consultas
– Planificadores específicos de bases de datos, planificadores de consultas específicos de bases de datos
– Comando EXPLAIN, uso de planes de ejecución de consultas
– Tipos de planificadores: ¿qué es un planificador de consultas de bases de datos?

R

RAG (véase generación aumentada por recuperación (RAG))
Réplicas de lectura, uso de estrategias de escalado
– (véase también almacenamiento en caché; estrategias de escalado)
lectura de código, lectura decódigo: resumen
– diagramas, extracción del código, arqueología del software
– entornos de desarrollo integrados (IDE), aprovechamiento de las funciones de los IDE:visualización de la estructura del código
– herramientas de gestión de código fuente, arqueología del software
Reeves, Jack, ¿Qué es el modelado de software y por qué lo hacemos?
– (véase también el código como diseño)

Refactorización segura, Refactorizaciónsegura: añadir pruebas antes de la refactorización
– (véase también pruebas automatizadas; sistemas desconocidos, modificación segura)
– Añadir pruebas antes, Añadir pruebas antes de refactorizar
pruebas de regresión, conduce a la coherencia y la repetibilidad
trabajo remoto, Comprender los retos deldesarrollo de software:retos del desarrollo de software remoto
– (véase también equilibrio entre vida laboral y personal)
desarrollo basado en el currículum, adquisición deliberada de habilidades
– (véase también adquisición de habilidades, deliberada)
Generación aumentada por recuperación (RAG), bases de datos vectoriales
– (véase también bases de datos, tipos de, bases de datos vectoriales)
Dependencias en tiempo de ejecución frente a dependencias en tiempo de compilación, compensaciones
– (véase también arquitectura de software)

S

SBOM (véase lista de materiales de software (SBOM))
Estrategias de escalado, uso de estrategias de escalado
– escalado horizontal, uso de estrategias de escalado
– Fragmentación, uso de estrategias de escalado
– escalado vertical, uso de estrategias de escalado
regla scout, la regla scout: cambios pequeños y reversibles
SDLC (véase ciclo de vida del desarrollo de software (SDLC))

Segunda ley de la arquitectura de software, registros de decisiones arquitectónicas
Sección 508, Ley de Rehabilitación, Diseño para todos
  – (véase también accesibilidad (a11y))
Capa de sockets seguros (SSL), Protección de las comunicaciones con HTTPS
Seguridad,Requisitos esenciales de seguridad:requisitos decumplimiento
  – Autenticación y autorización, Buenas prácticas deautenticación: buenas prácticas deautenticación
    – Autenticación multifactor, Buenas prácticas de autenticación
    – Gestión de contraseñas, buenas prácticas de autenticación
    – Protección de cuentas de usuario, Buenas prácticas de autenticación
  – Cumplimiento normativo,Requisitos de cumplimiento normativo: requisitos decumplimiento normativo
  – Cifrado, Cifrado: tu última línea de defensa
  – HTTPS (HTTP Secure), Protección de las comunicaciones conHTTPS: Protección de las comunicaciones con HTTPS
    – (véase también Seguridad de la capa de transporte (TLS))
  – Datos de usuario, protección, Protección de los datos de usuario
    – (véase también información de identificación personal (PII))
Búsqueda semántica, bases de datos vectoriales
  – (véase también bases de datos, tipos de, bases de datos vectoriales)
desplazamiento hacia la izquierda, no resuelvas demasiado rápido
proyectos paralelos, Explorar diferentes ámbitos y tecnologías:experimentar con proyectos paralelos

– (véase también gestión de la carrera profesional, planificar tu trayectoria)
Búsqueda por similitud, bases de datos vectoriales
– (véase también bases de datos, tipos de, bases de datos vectoriales)
Principio de responsabilidad única (SRP), mejora la mantenibilidad
Cambios pequeños y reversibles, cambios pequeños y reversibles:solicitudes de extracción y revisiones de código
– (véase también confirmaciones atómicas)
Habilidades sociales, Dominar las habilidades sociales en el mundo tecnológico: resumen
– Comunicación colaborativa, Comunicación colaborativa
– El código como medio de comunicación, El código como medio de comunicación
– (véase también escribir código)
– continuo de comunicación, canales de comunicación
– Operador empresarial,Operador empresarial: operador empresarial
– conocer a tu público, conoce a tu público
– reuniones,reuniones-reuniones
– mensajería (Slack, Teams, etc.),mensajería-mensajería
– Presentaciones,Presentaciones-Presentaciones
– influencia, practicar, Practicarinfluencia-gestión de las partes interesadas
– articular el valor empresarial, comprender y articular el valor
– matriz de poder/interés, gestión de las partes interesadas

– gestión de las partes interesadas,gestión de las partesinteresadas-gestión de las partes interesadas
– Enfoques estratégicos, Enfoques estratégicos para influir-Enfoques estratégicos para influir
– gestión del tiempo,gestión del tiempo:mantenerse centrado en las tareas
– Desarrollo impulsado por la hamaca, gestión del tiempo
– Calendario del creador, Calendario del creador
– técnica Pomodoro, mantenerte centrado en la tarea
Arqueología de software, Abordarcódigo desconocido:arqueología de software
– Creación de modelos mentales (véase Modelos mentales)
Arquitectura de software,Arquitectura de software: resumen
– (ver también estilos arquitectónicos)
– Katas arquitectónicos, ponerlo en práctica
– Consideraciones sobre ¿Qué es la arquitectura?
– versus diseño, Arquitectura versusdiseño: Arquitectura versus diseño
– Arquitectura evolutiva, El arquitecto ágil
– Como decisiones fundamentales, ¿Qué es la arquitectura?
– El problema del rendimiento, funciones de aptitud
– decisiones estratégicas frente a tácticas, Arquitectura frente a

– compensaciones en ¿Qué es la arquitectura?-Compensaciones
lista de materiales de software (SBOM), requisitos de cumplimiento
– (véase también gestión de dependencias)

ciclo de vida del desarrollo de software (SDLC), Un ingeniero con cualquier otro nombre, Por qué las revisiones de código son más importantes que nunca
Ingeniero de software, de programador a ingeniero
– definición de, de programador a ingeniero
– frente a programador y desarrollador, de programador aingeniero: un ingeniero con cualquier otro nombre
modelado de software,modelado: ¿qué es el modelado de software y por qué lo hacemos?
– (véanse también los diagramas)
Soluciones, prematuras, no soluciones demasiado rápido
– (véase también Cinco porqués; problemas de conceptos erróneos)
herramientas de gestión de código fuente, arqueología del software
– para la lectura de código, Arqueología del software
Spring Boot, monitoreo y análisis de bases de datos
Documentación de Spring REST, pruebas como documentación
– (véase también documentación)
Spring Security, buenas prácticas de autenticación
Inyección SQL, sentencias preparadas
– (véase también sentencias preparadas)
SRP (véase principio de responsabilidad única (SRP))
SSL (véase capa de sockets seguros (SSL))
Decisiones estratégicas frente a tácticas, arquitectura frente a diseño
– (véase también arquitectura de software frente a diseño)
SUT (véase sistema bajo prueba (SUT))

Mantenimiento del sistema,mantenimiento del sistema:tomar medidas
– gestión de dependencias, gestión de dependencias
– (véase también lista de materiales de software (SBOM))
sistema bajo prueba (SUT), redacción de pruebas unitarias

T

Desarrollo en forma de T, desarrollo en forma de T
TDD (véase desarrollo basado en pruebas (TDD))
Deuda técnica, El reto de trabajar con código existente
Radar tecnológico, ¿Dónde deberías invertir tu tiempo?,Aprendizaje estratégico:crea un radar tecnológico personal
– (véase también adquisición de habilidades, deliberada)
– Crea tu propio radar (BYOR), Crea un radar tecnológico personal
desarrollo basado en pruebas (TDD), primeros pasos
– Ciclo rojo-verde-refactorización, Introducción
Refactorización basada en pruebas (véase Refactorización segura, añadir pruebas antes)
Enfoque de prueba final, Introducción
– (véase también desarrollo basado en pruebas (TDD))
Pruebas (véase Pruebas automatizadas; Pruebas manuales; Redacción de pruebas)
Pruebas como documentación, Lectura de pruebas paraobtener información: descubrimiento de casos en perímetro y límites, El problema con los comentarios en el código, Actúa como documentación
– (véase también documentación)

– descubrimiento de casos perimetrales, Descubrimiento de casos perimetrales y límites
– documentación viva, Pruebas como documentación viva
Seguridad de la capa de transporte (TLS), proteger la comunicación con HTTPS
Modelo Twin Peaks, Arquitectura frente a diseño
– (véase también arquitectura de software frente a diseño)

U

UI (véase diseño de interfaz de usuario (UI))
UML (véase Lenguaje unificado de modelado (UML))
Sistemas desconocidos, comprenderbases de código desconocidas:solicitudes de extracción y revisiones de código
– Modificación segura, Realizar cambiosde forma segura: solicitudes de extracción y revisiones de código
– (véase también regla scout)
– Cambios pequeños y reversibles, Cambios pequeños y reversibles:solicitudes de extracción y revisiones de código
– (véase también «commits atómicos»)
– Proceso de ejemplo para explorar, un proceso de ejemplo
– Comprensión, Comprensión de bases de código desconocidas:ampliación gradual de tu ámbito
– (véase también leer código)
– Dependencias externas, Localización de dependencias externas
– marcos internos (de desarrollo propio), Localización de marcos ybibliotecas internos:trabajo con marcos internos
lenguaje de modelado unificado (UML), ¿Qué diagramas necesitas?

Usabilidad, Diseño para todos:¿qué es la usabilidad?
– (véase también accesibilidad (a11y))
– Descubribilidad, ¿Qué es la usabilidad?, Haz que lo correcto sea lo obvio:haz que lo correcto sea lo obvio
– Eficiencia, ¿Qué es la usabilidad?
– Gestión de errores, ¿Qué es la usabilidad?
– (véase también mensajes de error)
– facilidad de aprendizaje, ¿Qué es la usabilidad?
– memorabilidad, ¿Qué es la usabilidad?
– Satisfacción del usuario, ¿Qué es la usabilidad?
Diseño de la interfaz de usuario (UI),Diseño de la interfaz de usuario: resumen
Perfiles de usuario, comprender el proyecto
Investigación de usuarios, ¿Qué son la localización y la internacionalización?-Impacto de la cultura
– Valores predeterminados, importancia de, tú no eres tu usuario:tú no eres tu usuario
– Principio «no des nada por sentado», conoce a tu usuario
– Impacto de la cultura, tú no eres tu usuario:impacto de la cultura
– (véase también internacionalización (I18N))
– Usuarios secundarios, conoce a tu usuario
– telemetría para, No eres tu usuario
UTC (Tiempo Universal Coordinado), Datos de fecha y hora
– (véase también datos de fecha y hora)

V

incrustaciones vectoriales, bases de datos vectoriales

– (véase también bases de datos, tipos de, bases de datos vectoriales)
buenas prácticas de control de versiones, buenasprácticas de control de versiones:solicitudes de extracción y revisiones de código,estrategias de control de versiones: el ciclo de vida del control de versiones
– Confirmaciones atómicas, Confirmar estratégicamente
– (véase también cambios pequeños y reversibles)
– nombres de ramas, ramas de apoyo
– Estrategias de ramificación, ramificar con prudencia
– Mensajes de confirmación, Confirma estratégicamente
– confirmaciones frecuentes, Confirma estratégicamente
– git blame, arqueología de software
– Git Flow,estrategias de control de versiones:el ciclo de vida del control de versiones
– ramas de hotfix, ramas de soporte
– Desarrollo basado en troncos, estrategias de control de versiones
Codificación Vibe, lectura de código,reseñas de Vibe Code:la nueva mentalidad de los desarrolladores

W

La mentalidad de «siempre lo hemos hecho así», el valor de una nueva perspectiva
Williams, Robin, Principios del diseño
Escribir código, escribircódigo: resumen
– Código repetitivo, evitar, cuidado con el código repetitivo
– Código inteligente, evitar, Pruebas como documentación

– (véase también complejidad)
– Método de la pirámide invertida, Escribir código para ser leído
– bases de código más pequeñas, menos es más:prefiere métodos cortos
escritura de pruebas,Escribir pruebas: escribirpruebas de extremo a extremo
– pruebas de extremo a extremo (E2E), escribir pruebas de extremo a extremo
– pruebas de integración, escribir pruebas de integración
– Pruebas unitarias, escribir pruebas unitarias:simulación

X

Definición de esquema XML (XSD), XML

Z

Notación Z, ¿Qué diagramas necesitas?
Ley cero de la informática, menos es más
– alta cohesión, la ley cero de la informática
– Bajo acoplamiento, La ley cero de la informática